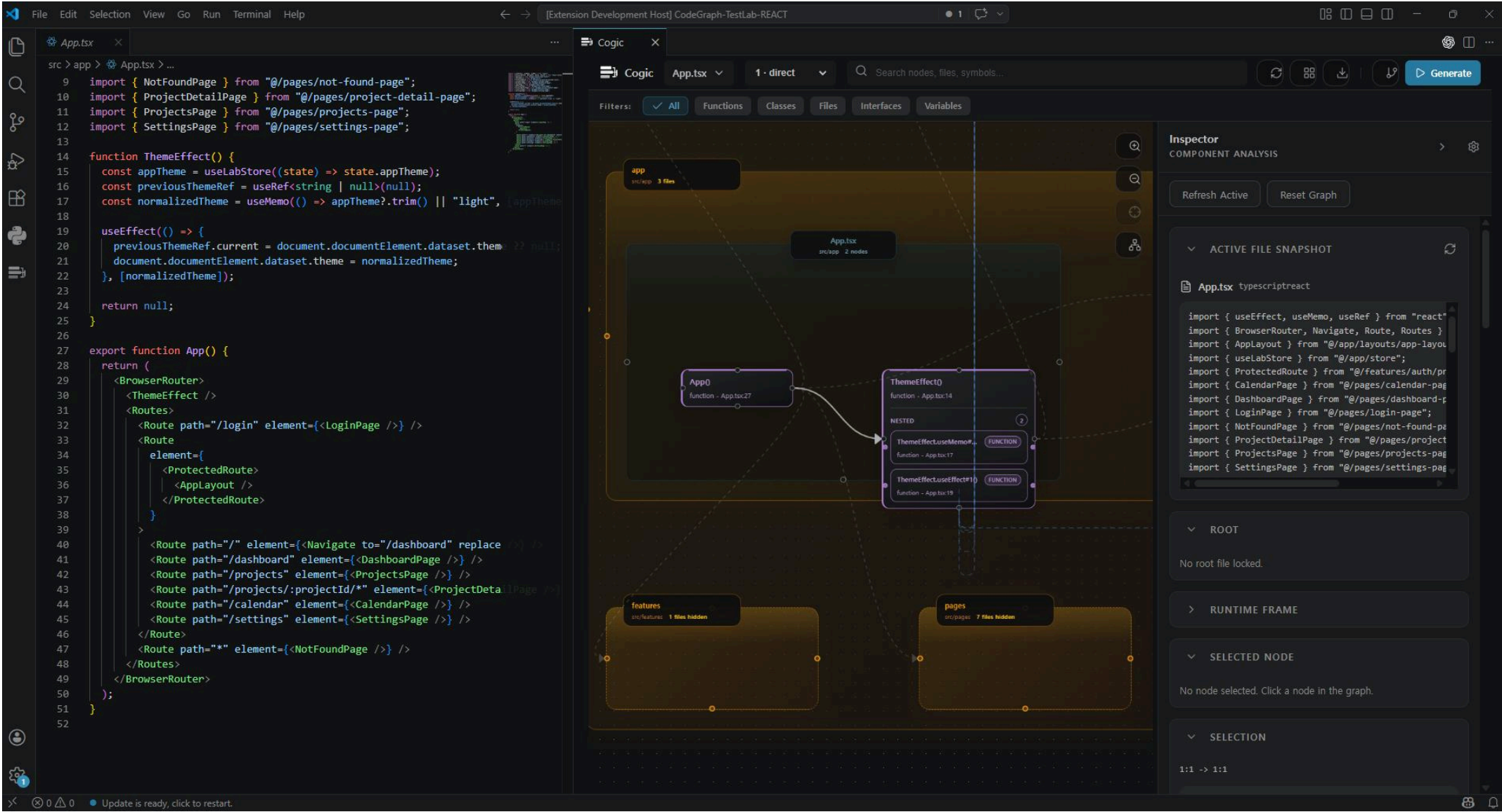




서비스

TypeScript·JavaScript 코드베이스를 정적 분석해
함수, 클래스, 타입, 호출, 참조, 상태 변경 및 데이터 흐름을
인터랙티브 그래프로 탐색할 수 있는 VS Code Extension

[GitHub](#)

[Microsoft Marketplace](#)

기술 스택

VS Code API TypeScript React
 ReactFlow Vite

역할

Planning & Design

코드 구조 탐색 문제 정의 및 기능 기획
그래프 기반 사용자 탐색 흐름 설계, UI/UX 디자인(figma)

Frontend & Extension Development

VS Code Extension 및 React Webview 개발
TypeScript Compiler API 기반 AST 분석
함수, 클래스, 타입 및 파일 관계 추출
호출 관계, 참조 관계, 데이터 흐름 그래프 구현
그래프 탐색 깊이와 필터 상태 관리
Webview 메시지 통신 및 상태 유지
VS Code Marketplace 배포 및 버전 관리

작업 기간

개발·운영 2026.02 - 현재

구성원

1인 프로젝트

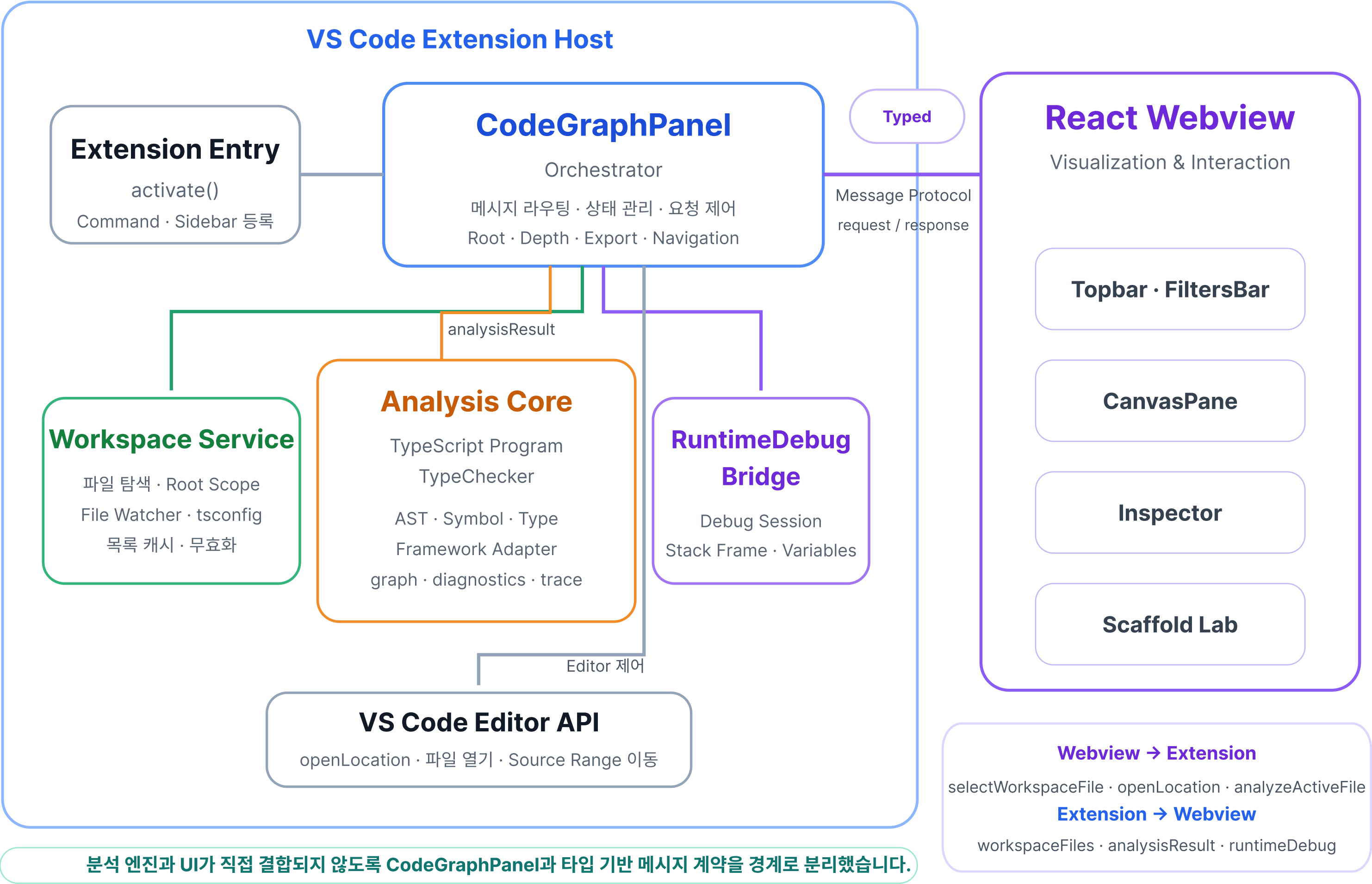
기획 UI/UX 설계 정적 분석 엔진 VS Code Extension
React Webview 배포 및 운영

프로젝트 회고

- TypeScript Compiler API의 Program·TypeChecker를 활용한 정적 분석 엔진 구현
- 코드 엔티티와 호출·참조·데이터 흐름을 독립적인 그래프 스키마로 설계
- React, Vue, Express, NestJS 분석을 Framework Adapter 구조로 확장
- Sidebar View와 Editor Panel 두 가지 실행 환경 지원
- 파일·폴더 Root 고정, 탐색 깊이 0~3단계, 외부 파일 확장 기능 구현
- JSON·JPG·SVG 내보내기 및 VS Code Marketplace v0.0.5 배포

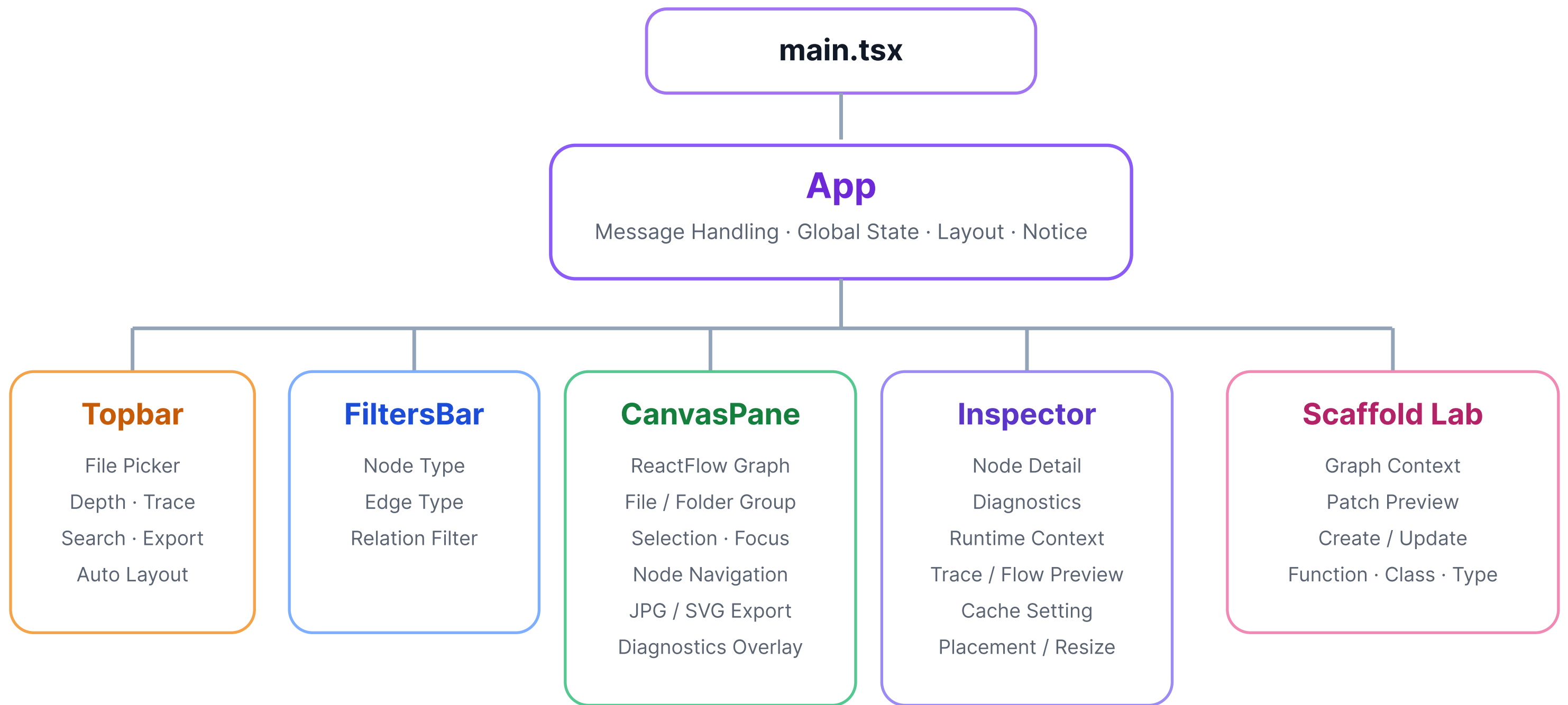
01. Extension-Webview 시스템 아키텍처

Extension Host가 분석과 VS Code API를 담당하고, React Webview는 시각화와 사용자 상호작용을 담당합니다.



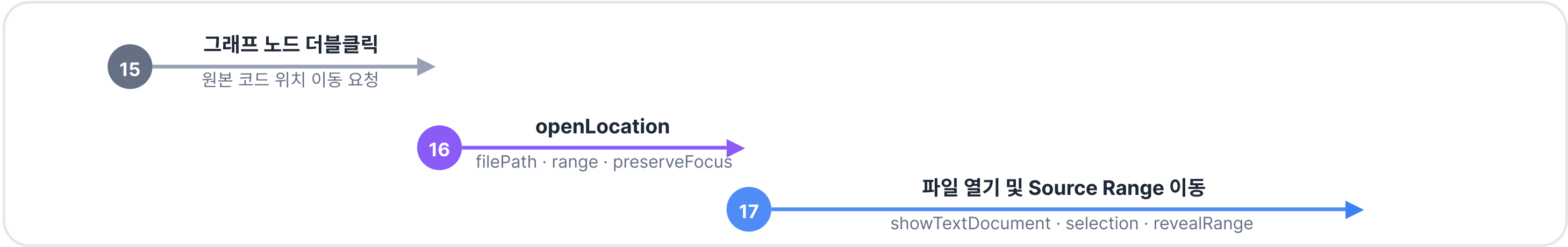
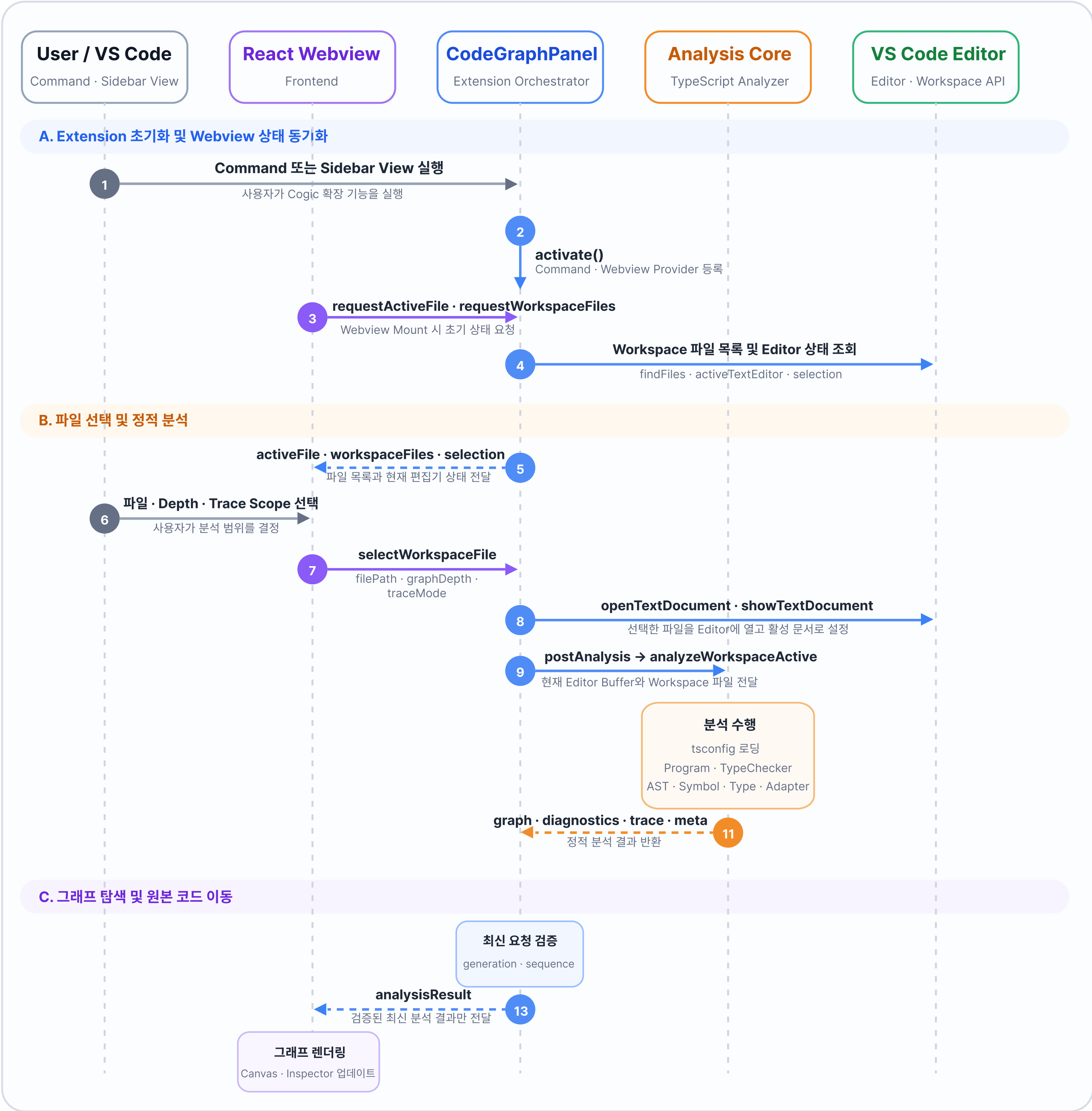
02. React Webview 컴포넌트 구조

App이 메시지·상태·레이아웃을 통합 관리하고, 그래프 렌더링과 상세 탐색 기능을 컴포넌트 단위로 분리했습니다.



03. Extension-Webview 메시지 흐름

워크스페이스 탐색, 정적 분석, 그래프 렌더링, 코드 위치 이동까지의 실제 요청·응답 흐름입니다.



04. 3단계 캐시와 성능 최적화

반복 분석 비용을 줄이되, 저장되지 않은 편집 내용과 파일 변경을 정확하게 반영하도록 캐시 경계를 설계했습니다.

10초
Workspace 목록 TTL

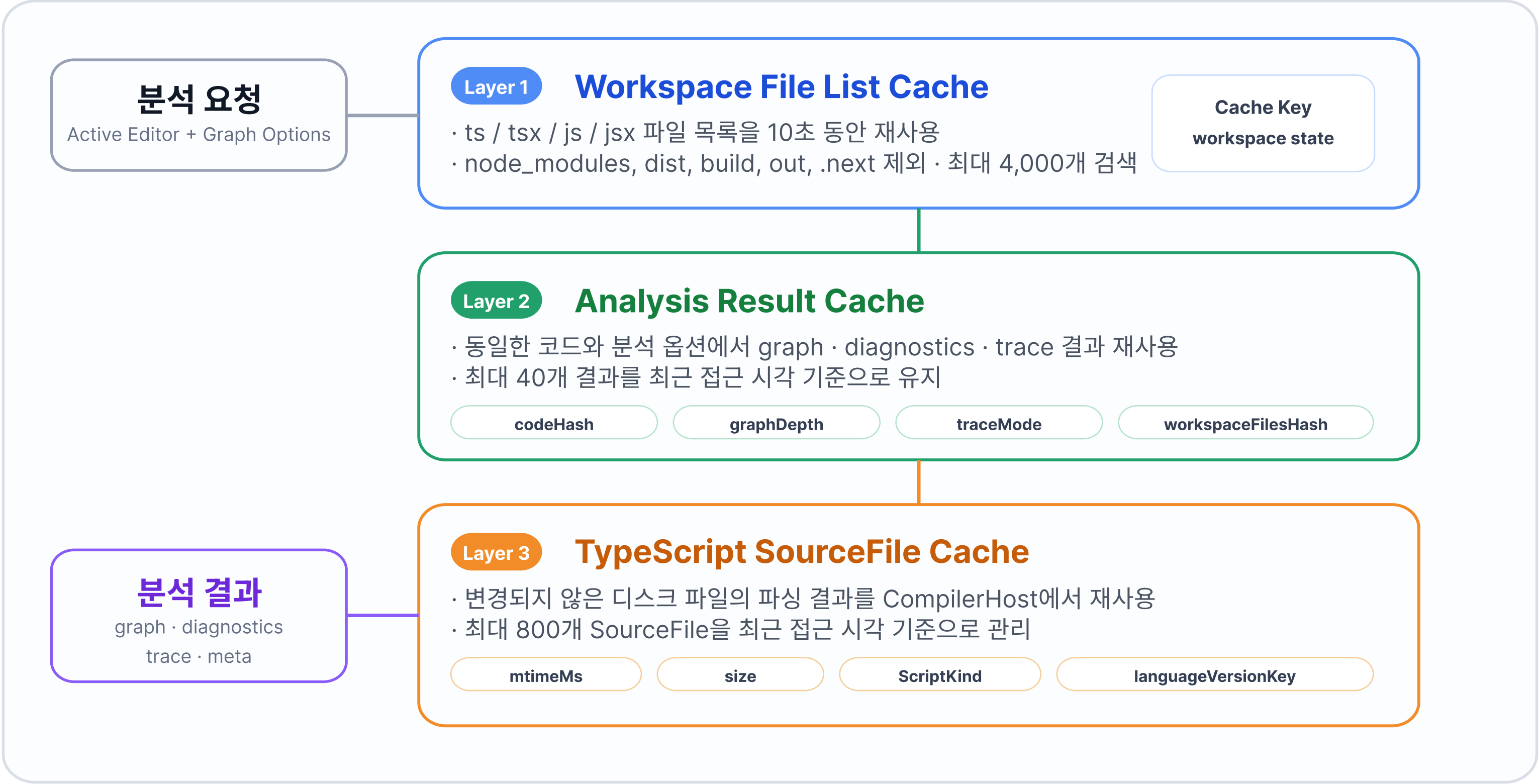
4,000
최대 검색 파일 수

40
분석 결과 캐시 수

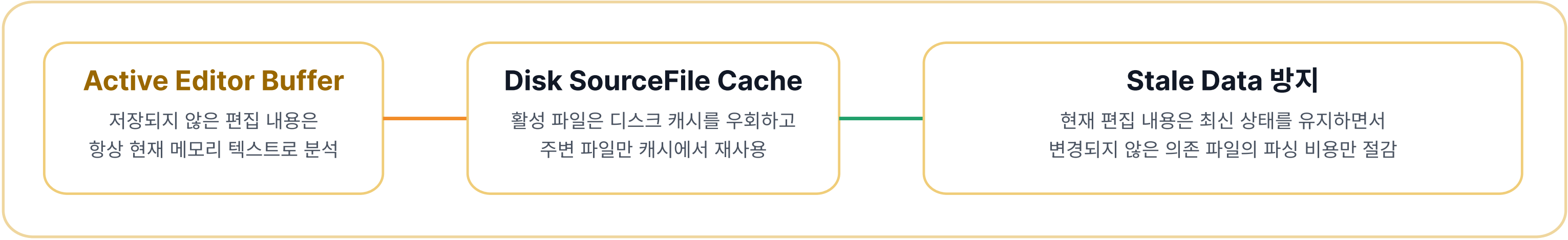
800
SourceFile 캐시 수

캐시 계층 구조

각 계층은 서로 다른 비용을 줄이며, 상위 캐시가 실패하면 하위 계층의 재사용 여부를 판단합니다.



정확성을 보장하는 예외 처리



캐시 무효화 조건

파일 시스템과 설정 변경을 감지해 분석 결과 캐시와 SourceFile 캐시를 명시적으로 비웁니다.

문서 내용 변경
onDidChangeTextDocument

문서 저장
onDidSaveTextDocument

파일 생성 · 삭제 · 이름 변경
FileSystemWatcher

tsconfig 변경
create · change · delete

캐시 설정 변경
analysisCache.enabled

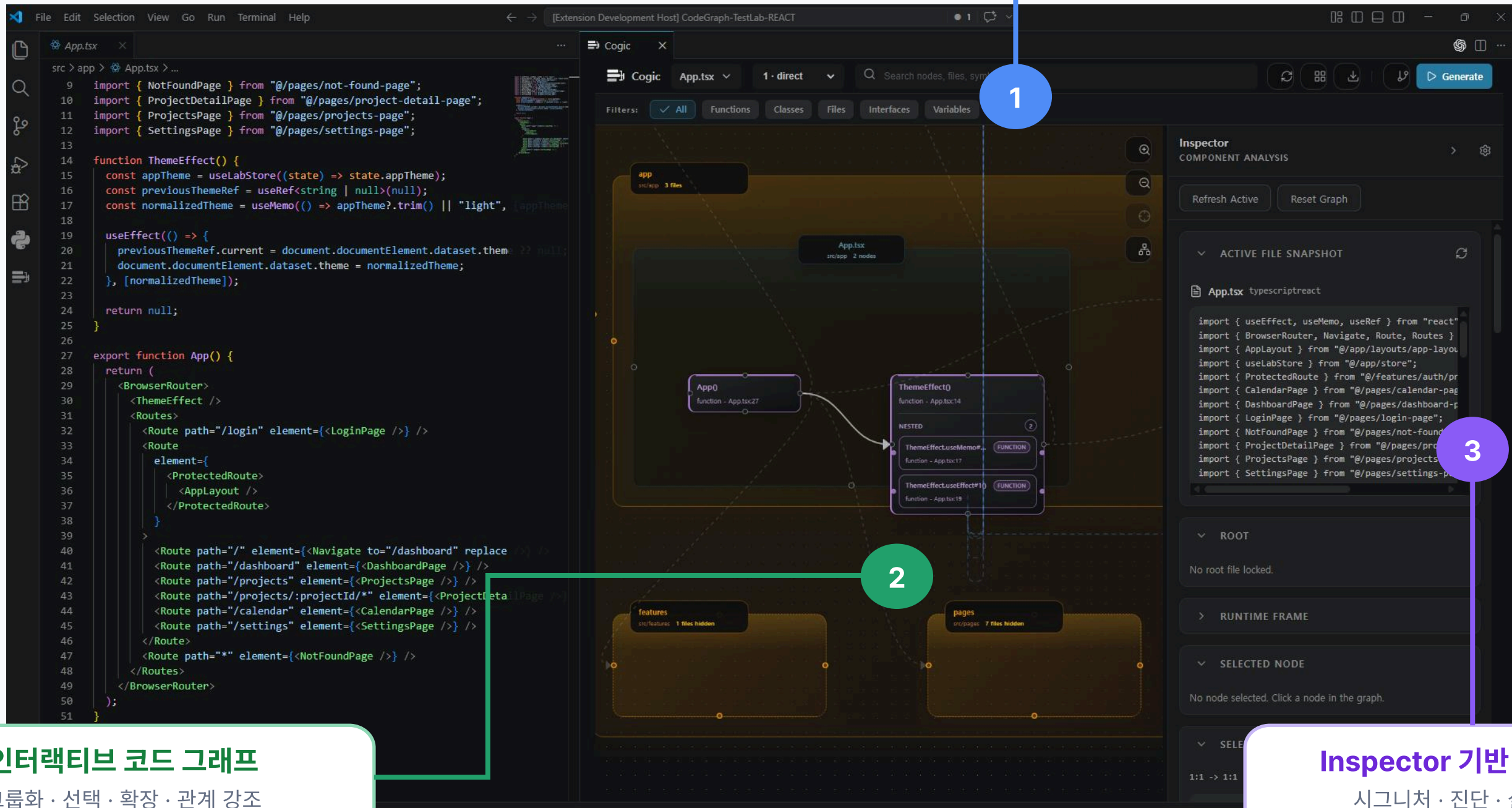
캐시 적중률만 높이는 것이 아니라, 변경 감지와 메모리 버퍼 우회를 함께 설계해 성능과 정확성을 동시에 확보했습니다.

05. 분석 결과를 탐색 가능한 개발 도구로 구현

정적 분석 결과를 단순 출력하지 않고, 그래프 탐색·코드 이동·실행 문맥·결과 활용으로 연결했습니다.

탐색 범위와 그래프 상태 제어

File · Depth · Filter · Trace · Export



인터랙티브 코드 그래프

그룹화 · 선택 · 확장 · 관계 강조

Inspector 기반 상세 탐색

시그니처 · 진단 · 실행 문맥

주요 사용자 경험

사용자가 코드 구조를 발견하고, 원본 코드로 이동하며, 분석 결과를 다시 활용하는 흐름으로 설계했습니다.

EXPLORE

그래프 생성 및 범위 탐색

파일과 연결된 호출·참조·데이터 흐름을 시각화하고 Depth와 Root Scope를 조절해 필요한 범위만 확장합니다.

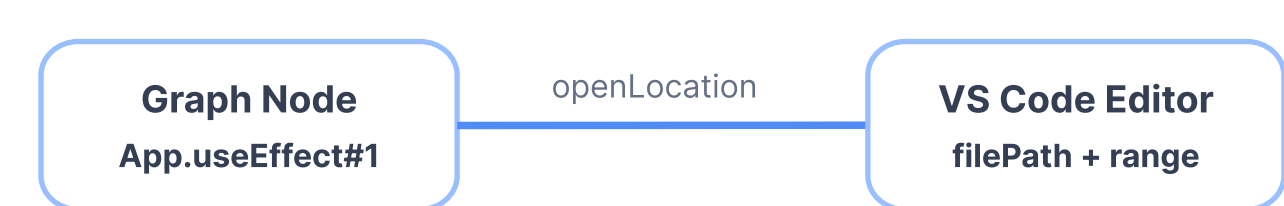


노드 선택 · 다중 선택 · 검색 · 관계 필터 · 외부 파일 확장

NAVIGATE

그래프와 원본 코드 양방향 연결

노드를 선택하면 Inspector가 갱신되고, 더블클릭하면 Extension이 파일을 열어 해당 Source Range로 이동합니다.

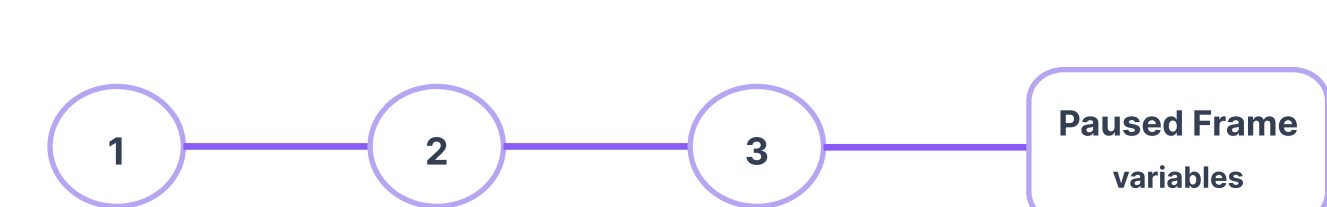


Graph → Inspector → Editor가 하나의 탐색 흐름으로 동작

UNDERSTAND

Trace와 Runtime Debug로 실행 문맥 확인

Trace Mode는 그래프 생성 순서를 단계별로 재생하고, Runtime Debug는 중단된 Stack Frame을 노드와 연결합니다.



정적 구조와 실제 실행 위치를 동일한 그래프 문맥에서 확인

REUSE

분석 결과 내보내기과 구조 생성

그래프와 상태를 JSON으로 저장하고, 시각 결과를 JPG·SVG로 내보내며 Scaffold Lab에서 코드 구조를 생성합니다.



분석 결과를 문서화하고 새로운 코드 구조 생성에 재사용